Lecture 16

Deep Reinforcement Learning

COMP532 – Machine Learning and Bioinspired Optimisation

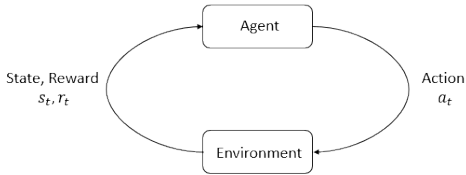
Meng Fang, University of Liverpool

Reinforcement Learning Setup

Key components:

- state s
- action a
- reward r
- policy $\pi(a|s)$

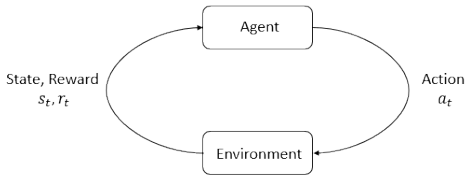
Goal:
maximize cumulative reward.



Agent-Environment Interaction

Loop:

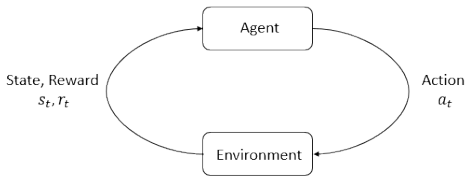
1. observe state s
2. choose action a
3. receive reward r
4. transition to new state s'



Markov Decision Process

An RL problem is often modeled as a:
Markov Decision Process (MDP)
Defined by:

- states S
- actions A
- transition probabilities
- reward function



Return

The goal is to maximize cumulative reward.

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + ...$$

γ = discount factor.

Example
Rewards over time: $[1, 1, 1]$, with $\gamma = 0.9$

$$G_0 = 1 + 0.9 \times 1 + 0.9^2 \times 1 = 2.71$$

Value Functions

State value function

$$V(s) = \mathbb{E}[G_t | S_t = s]$$

Expected return starting from state s.

Action value function

$$Q(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a]$$

Expected return taking action a in state s.

Example
Gridworld game:
$V(s) = 5$ means that starting from state s, the agent expects a total future reward of 5.
$Q(s, \text{right}) = 6$ means taking action right from s leads to higher expected return.

Bellman Equation

The Bellman equation defines recursive relationships.

$$Q(s, a) = r + \gamma \max_{a'} Q(s', a')$$

Example
$r = 1$, $\gamma = 0.9$, and $\max_{a'} Q(s', a') = 5$

$$Q(s, a) = 1 + 0.9 \times 5 = 5.5$$

Q-Learning

Q-learning updates estimates using:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left(r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right)$$

Example

$Q(s,a) = 2$, $r = 1$, $\gamma = 0.9$, $\max_{a'} Q(s',a') = 5$, $\alpha = 0.1$

$$\text{target} = r + \gamma \max_{a'} Q(s',a') = 1 + 0.9 \times 5 = 5.5$$

$$\text{error} = \text{target} - Q(s,a)$$

$$Q(s,a) \leftarrow 2 + 0.1(5.5 - 2) = 2.35$$

Exploration vs Exploitation

Agents must balance:

- exploration
- exploitation

Example
Choosing a restaurant:

- Exploitation: go to your favourite restaurant.
- Exploration: try a new restaurant that might be better.

Common strategy:
ϵ-greedy policy.

Deep Q-learning / Deep Q Networks

When state spaces are large:

$$Q(s, a) \approx Q_\theta(s, a)$$

A deep neural network approximates the Q-function.

Example
In Atari games, the state is the raw game screen (pixels).
Instead of a Q-table, a neural network predicts Q-values from the image.

Training Deep Q Networks

We train the neural network using the Bellman target.

$$y = r + \gamma \max_{a'} Q(s', a'; \theta^-)$$

Loss function:

$$L(\theta) = (y - Q(s, a; \theta))^2$$

Target Network (stabilise training)
A separate frozen network with parameters θ^- is used to compute the target.
It is periodically updated from the main network θ.

Gradient descent updates the parameters θ.

Experience Replay

Store experiences:

$$(s, a, r, s')$$

Train using random mini-batches.

Example
Trajectory collected from the environment:

$$(s_1, a_1, r_1, s_2), (s_2, a_2, r_2, s_3), (s_3, a_3, r_3, s_4), \cdots$$

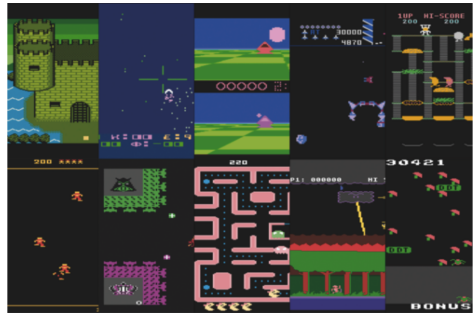
Training samples are drawn randomly from the replay buffer.

Benefits:

- stabilizes learning
- improves data efficiency

Deep Q Networks: Case Study

Example: Atari games
Objective: Complete the game with the highest score
State: raw game frames (images)
Action: joystick commands
Reward: Score increase/decrease at each time step

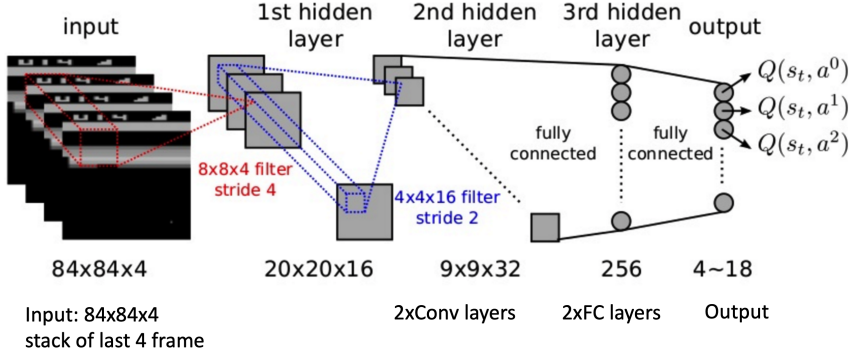


A convolutional neural network predicts Q-values for each action from the game screen.

$$Q(s, a) \approx Q_\theta(s, a)$$

The agent learns to play directly from pixels.
Google DeepMind's Deep Q-learning:
https://www.youtube.com/watch?v=V1eYniJ0Rnk

Deep Q Networks: Atari Case Study



- **State:** $84 \times 84 \times 4$ stack of recent frames (after grayscale and down-sampling)
- **Network:** Convolutional layers extract visual features from the game screen
- **Output:** The network predicts $Q(s, a)$ for each possible action
- **Policy:** Select the action with the highest value

$$a^* = \arg\max_a Q(s, a)$$

From Q-Learning to Modern Deep RL

Tabular Q-Learning

- Learn action values using the Bellman update
- Store values in a table: $Q(s, a)$
- Works for small state spaces

Deep Q Networks (DQN)

- Replace the Q-table with a neural network
- $Q(s, a) \approx Q_\theta(s, a)$
- Key techniques:
  - Experience replay
  - Target networks

Modern Deep RL

- Policy gradient methods
- Actor–critic algorithms
- Applications in robotics, games, and large-scale decision making

Reinforcement Learning Applications

RL is used in many real-world domains:

- **Robotics:** learning locomotion and manipulation
- **Autonomous driving:** decision making and control
- **Recommender systems:** long-term user engagement optimisation
- **Game AI:** superhuman agents in complex games

RL and AI Agents

Reinforcement learning provides a framework for agents to:

- learn behaviours through interaction with the environment
- adapt to changing situations
- improve decisions using reward feedback

Example: an Atari agent learns to maximise game score through trial and error.

Next Lecture

Next:

- LLM agents
- reasoning and tool use
- AI systems built on language models

Key Takeaways

- RL studies learning through interaction.
- Value functions estimate future reward.
- Q-learning updates estimates via Bellman equation.
- Deep Q Networks combine RL with neural networks.